

JSON

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is JSON?

- **JSON** = JavaScript Object Notation
- Lightweight, text-based format for storing and exchanging data
- Inspired by JS object literals; now a language-independent standard
- The format most web APIs speak; used in config files and `localStorage`
- Core idea: **serialize** (value to text) and **parse** (text back to value)

What JSON Looks Like

```
{  
  "name": "Alice",  
  "age": 30,  
  "isAdmin": false,  
  "roles": ["editor", "reviewer"],  
  "address": { "city": "Seattle", "zip": "98101" },  
  "manager": null  
}
```

- Looks like a JS object, but it is **text**, not live code, with stricter rules

JSON Value Types

- **string**: always double quotes: `"hello"`
- **number**: `42`, `3.14`, `-7` (no `NaN`, no `Infinity`)
- **boolean**: `true` or `false`
- **null**: `null`
- **object**: `{ "key": value, ... }`
- **array**: `[ value, value, ... ]`
- Objects and arrays nest to any depth

Rules That Trip People Up

```
{
  name: "Alice",           // keys must be in double quotes
  'city': 'Seattle',       // single quotes not allowed
  "age": 30,               // no trailing comma
  "greet": function () {}, // no functions
  "missing": undefined,    // no undefined
  // comments not allowed either
}
```

- Double quotes on every key/string, no trailing commas, no comments
- No functions, `undefined`, `NaN`, or `Infinity`; dates are written as strings

JSON vs XML

```
{ "user": { "name": "Alice", "roles": ["editor", "reviewer"] } }
```

```
<user>  
  <name>Alice</name>  
  <roles><role>editor</role><role>reviewer</role></roles>  
</user>
```

- JSON is **less verbose**: no repeated closing tags
- JSON has **native arrays**; parses straight into JS objects
- XML still wins for attributes, namespaces, mixed markup

JSON.stringify: Object to Text

```
const user = { name: "Alice", age: 30, isAdmin: false };

const text = JSON.stringify(user);
console.log(text);
// {"name":"Alice","age":30,"isAdmin":false}

console.log(typeof text); // "string"
```

- `JSON.stringify(value, replacer, space)`
- Compact form for storing/sending data

Pretty-Printing with **space**

```
const user = { name: "Alice", roles: ["editor", "reviewer"] };

console.log(JSON.stringify(user, null, 2));
// {
//   "name": "Alice",
//   "roles": [
//     "editor",
//     "reviewer"
//   ]
// }
```

- Third argument: number of spaces (or a string) per indent level
- Use the pretty form for logs and files people read

Filtering with **replacer**

```
const user = { name: "Alice", age: 30, password: "s3cret" };

// Array of keys to keep:
JSON.stringify(user, ["name", "age"]);
// {"name":"Alice","age":30}

// Function: return undefined to drop a key:
JSON.stringify(user, (key, value) =>
  key === "password" ? undefined : value);
// {"name":"Alice","age":30}
```

- Handy for stripping sensitive fields before sending data

Customizing with `toJSON`

```
const event = { title: "Launch", when: new Date("2026-07-07T15:00:00Z") };
JSON.stringify(event);
// {"title":"Launch","when":"2026-07-07T15:00:00.000Z"}

class Money {
  constructor(cents) { this.cents = cents; }
  toJSON() { return (this.cents / 100).toFixed(2); }
}
JSON.stringify({ price: new Money(1999) }); // {"price":"19.99"}
```

- If a value has `toJSON()`, stringify serializes its result; this is how `Date` works

JSON.parse: Text to Object

```
const text = '{"name":"Alice","roles":["editor","reviewer"]}';

const user = JSON.parse(text);
console.log(user.name);      // "Alice"
console.log(user.roles[0]); // "editor"
```

- Invalid JSON **throws**; wrap untrusted input in `try/catch`

```
try {
  JSON.parse("{ not valid }");
} catch (error) {
  console.log("Could not parse:", error.message);
}
```

Transforming with `reviver`

```
const text = '{"title":"Launch","when":"2026-07-07T15:00:00.000Z"}';
const isoDate = /^\\d{4}-\\d{2}-\\d{2}T/;

const event = JSON.parse(text, (key, value) =>
  typeof value === "string" && isoDate.test(value)
    ? new Date(value)
    : value);

console.log(event.when instanceof Date); // true
```

- `reviver` runs for every key/value pair; its return becomes the value
- Classic use: revive ISO strings back into real `Date` objects

Common Pitfalls

```
// Dates come back as strings:
const round = JSON.parse(JSON.stringify({ when: new Date() }));
round.when instanceof Date; // false - it is a string

// Circular references throw:
const node = { name: "A" };
node.self = node;
JSON.stringify(node); // TypeError: Converting circular structure

// undefined / functions silently disappear:
JSON.stringify({ a: undefined, b: () => {}, c: 1 }); // {"c":1}
JSON.stringify([undefined, function () {}, 1]);      // [null,null,1]
```

Where JSON Is Used

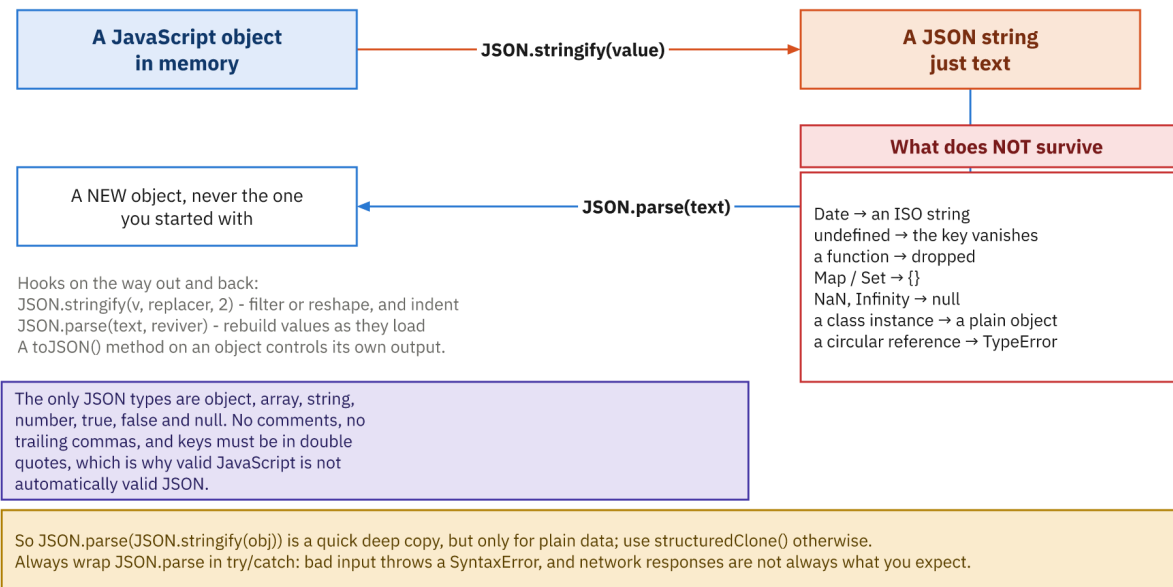
- **Web APIs:** most REST APIs speak JSON; `fetch` gives `response.json()`
- **Config files:** `package.json`, `tsconfig.json`, and many tool configs
- **Browser storage:** only holds strings, so stringify in, parse out

```
localStorage.setItem("settings", JSON.stringify({ theme: "dark" }));  
const saved = JSON.parse(localStorage.getItem("settings"));  
console.log(saved.theme); // "dark"
```

Out and Back Again

The JSON Round Trip

JSON is TEXT. Going out and coming back is not lossless: know what does not survive.



- Object to text and back, but the trip is not lossless
- Dates become strings, **undefined** vanishes, circular references throw